

Resenha Crítica: Artigo 2 — On the Criteria to Be Used in Decomposing Systems into Modules

Gustavo Pessoa Firmino Duarte

8 de Março de 2026

1 Introdução

Publicado em 1972 na *Communications of the ACM*, o artigo de David L. Parnas, “On the Criteria to Be Used in Decomposing Systems into Modules”, é um marco fundamental da engenharia de software. Em uma época em que a modularização era frequentemente confundida com simples divisão de fluxo de execução, Parnas propõe uma crítica precisa e construtiva: o critério correto para decompor um sistema não é a sequência de passos computacionais, mas sim o *ocultamento de informação* (*information hiding*). Cinquenta anos depois, esse princípio continua sendo um dos pilares do design de software moderno, presente em arquiteturas orientadas a objetos, microserviços e APIs.

2 O Princípio do Ocultamento de Informação

Parnas utiliza o exemplo do sistema KWIC (*Key Word in Context*) para contrastar duas estratégias de modularização. A primeira decompõe o sistema seguindo o fluxo algorítmico: leitura de entrada, cálculo das deslocamentos circulares, ordenação e saída. Cada módulo corresponde a uma etapa do processo. A segunda decompõe com base no ocultamento de decisões de projeto: cada módulo esconde uma decisão que pode mudar — como a estrutura de dados usada para armazenar as linhas, o algoritmo de ordenação ou o formato de saída.

O autor demonstra que a primeira abordagem cria um acoplamento alto: qualquer mudança na representação interna dos dados exige alterações em múltiplos módulos. Já na segunda abordagem, cada decisão de projeto fica encapsulada, tornando as mudanças locais e invisíveis aos demais módulos. Parnas defende que um bom módulo deve ter uma *interface* estável que esconda seus detalhes internos, permitindo que a implementação evolua sem propagação de impacto.

Além da manutenibilidade, Parnas aponta outros benefícios do ocultamento de informação: compreensibilidade seletiva (é possível entender um módulo sem conhecer os demais), desenvolvimento independente por equipes diferentes e maior facilidade de testes unitários. Esses benefícios antecipam, de forma notável, os princípios do desenvolvimento ágil e da arquitetura orientada a componentes que dominariam as décadas seguintes.

3 Conexão com a Prática Profissional

A leitura deste artigo ressoa diretamente com minha experiência no desenvolvimento de um MVP de e-commerce utilizando Node.js, Prisma ORM, React e SQLite. Ao escolher o Prisma ORM como camada de acesso a dados, tomei — sem ainda ter lido Parnas — exatamente a decisão que ele recomenda: esconder do restante da aplicação os detalhes do banco de dados relacional subjacente. O Prisma age como um módulo no sentido parnesiano: os serviços de domínio apenas chamam métodos como `prisma.product.findMany()`, sem qualquer conhecimento de SQL, índices ou estrutura de tabelas.

Quando, em determinado momento do projeto, precisei migrar parte dos dados para uma estrutura diferente, apenas o schema do Prisma e as migrações precisaram ser alterados — as camadas de serviço e de apresentação permaneceram intocadas. Essa experiência concreta validou o argumento central de Parnas: sistemas decompostos por ocultamento de informação são substancialmente mais fáceis de modificar.

Também reconheço, em retrospecto, violações desse princípio. Em um projeto acadêmico de cinco dias, sob pressão de prazo, struturei vários componentes React que consumiam diretamente objetos do banco de dados, sem nenhuma camada de abstração. Qualquer alteração no modelo de dados exigia busca e substituição em dezenas de componentes — exatamente o problema descrito por Parnas na decomposição baseada em fluxo.

4 Conclusão

O artigo de Parnas é notavelmente atemporal. Sua proposta central — decompor sistemas com base nas decisões de projeto que cada módulo esconde — continua sendo o critério mais sólido que conheço para avaliar a qualidade de uma arquitetura. O princípio do ocultamento de informação não é apenas uma regra teórica; é uma ferramenta prática que reduz o custo de mudança, diminui o acoplamento e facilita o trabalho em equipe. Em um cenário de software cada vez mais distribuído e dinâmico, onde decisões tecnológicas mudam com frequência, a lição de Parnas torna-se ainda mais valiosa: projete para a mudança, não apenas para o funcionamento presente.